Hochschule
Bonn-Rhein-Sieg
University of Applied Sciences

Prof. Dr. Sascha Alda
Moritz Balg, Javed Razzaq, Luca Ringhausen und Team

20. Oktober 2026

Übungen zur Vorlesung Software Engineering I
WS 2026 / 2027

Übungsblatt Nr. 4
(Abgabe bis: Mittwoch, den 11.11.2026, **09:00 Uhr)**

Aufgabe 1 (Entwicklung von User Stories, Mind Map 10 Punkte):

Die neue Präsidentin der Hochschule hat uns (Ihnen!) einen Projektauftrag überreicht. Das Ziel soll es sein, eine Webanwendung zu entwickeln, die Studierende für Projektarbeiten zusammenbringt. KI unterstützt bei der Suche nach Teammitgliedern anhand von Interessen, Kompetenzen, Lernzielen und zeitlicher Verfügbarkeit (Projekt „TeamMatchAI@HBRS“). Die Software adressiert im Wesentlichen die Studierenden der Hochschule Bonn-Rhein-Sieg als wesentlicher (*primärer*) Stakeholder. Weitere Stakeholder (z.B. Dozenten) können natürlich einbezogen werden.

Ihre Aufgaben:
a.)
Die Präsidentin wünscht sich zunächst ein Mind Map mit potentiellen Ideen für eine solche Software. Dieses Mind Map können Sie z.B. mit dem freiverfügbaren Tool Mermaid entwickeln, welches auch im weiteren Verlauf der Veranstaltung immer mal wieder zum Einsatz kommt:

https://mermaid.live
(Hier gerne das Sample Diagramm vom Typ “Mindmap” verwenden und weiter-entwickeln. Ein Export als PDF ist möglich, falls man sich z.B. mit einem GitHub-Account anmeldet. Ansonsten Upload eines Screencast oder der textuellen Spezifikation).

Ihre Ideen sollten Sie mit Hilfe des Mind Map generell strukturieren. Exportieren oder speichern Sie ihr Mind Map aus dem Tool und laden Sie die Datei als Ergebnis via LEA hoch. Sie sollten auf erster Ebene *mindestens* 5 Ideen-Kategorien generieren und diese, auf zweiter Ebene, noch mal mit je *mindestens* 2-3 Unterideen verfeinern.

b.)
Es soll dann ihre nächste Aufgabe sein, mögliche Anforderungen an diese Software durch *mindestens* acht *aussagekräftige* User Stories niederzuschreiben, um die Ziele der Präsidentin zu konkretisieren. Gehen Sie wie folgt vor:

a.) Jede User Story sollte die in der Vorlesung eingeführten INVEST-Eigenschaften erfüllen. Jede Story sollte zu einem Epic zugeordnet sein.

b.) Geben Sie für jede User Story eine aus ihrer Sicht plausible Mehrwertschätzung sowie eine aus ihrer Sicht denkbare Aufwandsschätzung basierend auf einer Fibonnacci-Zahl (hier ggf. vorarbeiten, siehe Kapitel 3).

c.) Jede User Story sollte mit verifizierbaren Akzeptanzkriterien versehen sein. Formulieren Sie pro User Story mindestens zwei Akzeptanzkriterien.

Hinweis: Eine Implementierung ist also nun zu diesem Zeitpunkt noch *nicht* notwendig. Dokumentieren Sie die User-Stories in einem Tool wie Excel oder Word. Abgabe hier dann bitte auch als PDF.

Aufgabe 2 (Tool zur Verwaltung von User-Stories, 20 Punkte)

Das Unternehmen WeissNix GmbH möchte von Ihnen ein Tool zur Verwaltung von User Stories. Dazu gibt das Unternehmen folgende User Stories (*US*) vor:

US 1: „Als Entwickler möchte ich in einem Eingabedialog die *notwendigen* Daten zur Eingabe einer User Story eingeben. Das soll mir die Verwaltung von User Stories erleichtern."

- Folgende Daten sollen dazu berücksichtigt werden: eine *eindeutige* ID der User Story, ein kurzer schriftlicher Titel, ein kurzes Akzeptanzkriterium, sowie die vier Kennzahlen zur Berechnung der Priorisierung gemäß der Formel nach Gloger (Kapitel 3) sowie die Zuordnung zu einem Projekt. Details können Sie auslassen.
- Auch der resultierende Priorisierungswert sollte für eine User Story vom System berechnet und abgespeichert werden.
- Eine Eingabe von ungültigen Werten aus der Formel von Gloger (z.B. eine negative Zahl für ein Risiko) sollte unterbunden werden.

US 2: „Als Entwickler möchte ich eine Übersicht über die eingegebenen Daten aller User Stories in *tabellarischer Form* als Ausgabe erhalten können, damit ich einen guten Überblick bekomme, welche User Stories ich priorisiert umzusetzen habe."

- Die Struktur der Tabelle können Sie selbst wählen. Eine Sortierung nach *einer* Kennzahl sollte vorhanden sein.

US 3: „Alle Eingaben sollen *persistent* abgespeichert werden, so dass ich diese nach einem Neustart des Programms wieder einsehen kann."

<u>Anmerkungen zur Entwicklung:</u>
Weitere Details zu den User Stories, die vom Unternehmen noch genannt wurden:

- Alle Befehle sollen über eine Kommandozeile („Shell", „Terminal", „Console"; keine GUI!) eingegeben werden können. Struktur der Eingabe:

```
> befehl [parameter]
```

- Als Befehle sind vorgesehen:
  - `enter` (Eingabe einer User Story, nur Ablage in den RAM-Speicher, also in die Klasse `Container` (siehe Hinweis unten)),
  - `store` (Persistentes Abspeichern von User Stories aus einem Container-Objekt auf einen Datenträger (vgl. Übung Nr. 3-2),
  - `load` (Laden von User Stories von einem Datenträger in ein Container-Objekt),
  - `dump` (eine nach den berechneten Priorisierungen *sortierte* Ausgabe der User Stories inklusive aller eingegeben Angaben),
  - `exit` (Verlassen der Anwendung),
  - `help` (Anzeige aller möglichen Befehle)

- Es soll der Source Code der Klasse Container aus der Übung 2 bzw. 3 zur internen Abspeicherung der User Stories in dem RAM-Speicher *wiederverwendet* werden. Passen Sie dazu die Klasse entsprechend an (hier: Anpassung der internen Liste!). Auch die Klassen zur persistenten Speicherung von Objekten gemäß des Strategy Pattern soll verwendet werden. Verwenden Sie hier die Strategy `PersistenceStrategyStream`.

- Die Implementierung der Ausgabe („Dump“) der Liste soll *stream*-basiert unter Anwendung des Patterns „Filter Map Reduce“ erfolgen. Siehe dazu auch Beispiele und Erläuterungen in der Musterlösung der Klasse `Container` in der Aufgabe 2-2 (siehe LEA). Hinweis: es müssen nicht alle Verarbeitungsschritte „Filter“, „Map“ oder „Reduce“ zum Einsatz kommen.

Weitere Anforderungen hat die Firma *zunächst* nicht, ist aber gespannt auf einen ersten Java-Prototypen! Insgesamt haben Sie viele Freiheitsgrade bei der Entwicklung, worüber Sie selber entscheiden (Beispiel: Eingabe einer User Story mittels des Befehls `enter`).

Planung des Projekts mit einem Kanban-Board:
Definieren Sie für sich technische Aufgaben („Tasks“), die man zur Umsetzung dieses Projekts benötigt. Verwalten Sie die Tasks mit *unserem* Kanban-Board:

https://lea.hochschule-bonn-rhein-sieg.de/goto.php/fold/1935203

Sie haben dort bereits eine Sammlung von empfohlenen Tasks vorliegen, die Sie gerne um weitere Tasks ergänzen können. Führen Sie ein einfaches Controlling der Aufgaben durch, indem Sie die Aufgaben zu den einzelnen Spalten zuordnen („hinziehen“).

Software-Test:
Entwickeln Sie eine repräsentative Anzahl von Äquivalenzklassen und Testfällen, um ihre Anwendung als Ganzes („end-to-end“) im Rahmen von Akzeptanztests hinreichend

zu testen. Diese Testfälle sollten manuell durchgeführt werden, eine Test-Automatisierung ist hier *nicht* erforderlich. Zudem sollten Sie einzelne Methoden durch JUnit-Tests hinreichend testen (vgl. dazu auch die Lösungen aus der Aufgabe 3-2).

Finale Hinweise:
Bitte bereiten Sie bis zur nächsten Übungsstunde am 28.10.2026 (bzw. 2.11.26) einen *ersten* Prototyp vor, der aber nicht auf LEA hochgeladen werden muss. Zur *interaktiven* Klärung von möglichen Rückfragen stehen die Übungsgruppenleiter als Product Owner (PO), stellvertretend für den Kunden, gerne zur Verfügung. Die Gesamtabgabe erfolgt somit für diese Übung erst zum 11.11.2026 (bzw. 16.11.26).

Die Aufgabe 4-2 ist die Basis für die Anfertigung des (freiwilligen!) Semesterprojektes. Anforderungen dazu werden im November 2027 veröffentlicht.